Chapter 16 — Testing and TypeScript

Two habits separate code that *seems* to work from code you can change without fear. **Automated tests** prove behaviour by running your code and checking the results. **Static types** (TypeScript) catch whole classes of mistakes — typos, wrong argument types, forgotten `null` checks — before the code runs at all. They overlap only a little: types tell you *the shapes fit*; tests tell you *the answers are right*. This chapter teaches both with tools that ship with Node 24 and the TypeScript compiler.

What you'll learn

1. Why we test, kinds of tests, and the test-runner landscape (node:test, Vitest, Jest, Playwright).
2. Assertions with `node:assert/strict`.
3. Writing and running tests with `node:test`: `test`/`describe`/`it`, hooks, subtests, skip/todo/only, reporters, watch mode.
4. Async tests and the classic failures (wrong expectation, timeout, missing `await`).
5. Test doubles: fakes, stubs, spies, `mock.fn`, `mock.method`, fake timers, module mocks.
6. Snapshot tests and code coverage.
7. TypeScript: what it is, `tsc`, `tsconfig.json`, TypeScript 7.
8. Running `.ts` files directly with Node (type stripping) and its rules.
9. Types: primitives, arrays, tuples, object types, interfaces, unions, literals, `any` vs `unknown`.
10. Narrowing, discriminated unions and exhaustive checks.
11. Generics and utility types.
12. Classes, `as const`, `satisfies`, enums and their alternatives.
13. Tests written in TypeScript, and JSDoc types for plain JavaScript.

Before you start. You need functions (Chapter 4), classes (Chapter 9), modules and `package.json` scripts (Chapter 10), errors (Chapter 11), and `async`/`await` (Chapter 12). Testing React components (Testing Library) and TypeScript with React are in the React Handbook: ch11 and ch12.

The code. Everything is in `code/` — a tiny project with its own `package.json` (`"type": "module"`, scripts `test`, `coverage`, `typecheck`) and `tsconfig.json`. The modules under test live in `code/src/`. All outputs are real: **Node 24.11.1** and **TypeScript 7.0.2** (`tsc`). Test files are ordinary scripts too — running `node cart.test.mjs` executes its tests and exits with code 1 if any fail — so the book's runner executes them like every other program. Files that fail *on purpose* (`demo_failing.mjs`, `ts_errors.ts`, `jsdoc_errors.mjs`) or need a special flag (`module_mock_demo.mjs`) are marked `// tools_run: skip`, and the exact command to run them is in their first line and in the text. Timings in test output (the `(0.42ms)` parts) vary from run to run.

16.1 Why test, and what kinds of tests exist

What it is

An **automated test** is code that runs other code with known inputs and **asserts** (checks) that the result is what you expect. A **test runner** finds test files, runs them, and reports passes and failures. A **test suite** is a group of related tests.

Analogy: a checklist a pilot runs before every flight. It is boring, fast, and catches the one thing you'd otherwise forget.

Why it matters

- **Regression safety**: after a refactor or upgrade, the suite tells you in seconds whether something that used to work broke.

- **Design feedback**: code that's hard to test (hidden global state, hard-wired network calls) is usually hard to reuse.
- **Executable documentation** and a **CI gate**: the test command's exit code decides whether a pull request can merge.

Kinds of tests

Kind	What it exercises	Speed	Typical tool (JS)
Unit	One function/class/module in isolation; dependencies replaced by doubles	Milliseconds	node:test, Vitest, Jest
Integration	Real pieces together (HTTP handler + a real database in a container)	Tens of ms to seconds	Same runners + Testcontainers
End-to-end (E2E)	The whole app through its real interface (a browser clicking through pages)	Seconds	Playwright, Cypress
Static checks	No execution: types, lint rules	Fast	TypeScript, ESLint

The **test pyramid** advises many unit tests, fewer integration tests and a handful of E2E tests, because higher layers are slower and flakier. The **testing trophy** (popular in front-end) makes integration tests the fattest layer, for the most confidence per test. Both agree: static checks at the bottom, few E2E tests, and test **behaviour**, not implementation details.

Test runners in 2026

Runner	Strengths	Notes
`node:test` (built in)	No dependencies, fast, runs TypeScript via type stripping; mocks, fake timers, snapshots, coverage built in	Stable since Node 20; used here
Vitest	Jest-compatible API, Vite's config and transforms, browser mode	Common for Vite / React projects
Jest	Very large ecosystem	Older projects; ESM historically needed extra config
Playwright Test	Real browsers (Chromium, Firefox, WebKit), auto-waiting, tracing	E2E and component tests
Bun test / Deno test	Built into those runtimes (§15.12)	Jest-like APIs

The concepts (describe/it, hooks, assertions, mocks, snapshots) are nearly identical across all of them; §16.7 has a translation table.

> **TL;DR**
>
> Tests run your code and check results automatically. Unit tests are fast and isolated; integration tests check real pieces together; E2E tests drive the whole app. Test behaviour, keep most tests fast, and let CI run them on every change.

16.2 Assertions with `node:assert/strict`

What it is

An **assertion** is a check that throws an `AssertionError` when it fails. A test *fails* exactly when something inside it throws. `node:assert` is Node's assertion module; import the **strict** variant, `node:assert/strict`, in which `equal` means `===`-style comparison (actually `Object.is`) and `deepEqual` also compares types and prototypes. (The legacy, non-strict mode uses `==` — avoid it.)

Assertion	Passes when
`equal(actual, expected)`	`Object.is(actual, expected)` — same primitive value or **same object reference**
`deepEqual(actual, expected)`	Same structure and values, recursively, **type-strict** (`1` ≠ `'1'`), same prototypes
`notEqual` / `notDeepEqual`	The opposites
`ok(value)`	`value` is truthy
`match(string, regex)` / `doesNotMatch`	Regex matches / doesn't
`throws(fn, expected?)`	`fn()` throws; `expected` can be an error class, a regex for the message, a validation object `{ name, message }`, or a function
`rejects(promiseOrFn, expected?)`	The promise **rejects** — must be `await`ed
`doesNotThrow` / `fail(msg)`	Rarely needed (just call the code) / always fails
`partialDeepStrictEqual(actual, expected)`	`actual` contains at least the properties in `expected` (newer; check your Node version)

`code/01_assert_basics.mjs` runs each assertion and prints PASS or FAIL:

```
import assert from 'node:assert/strict';

function check(label, fn) {
  try { fn(); console.log('PASS', label); }
  catch (err) { console.log('FAIL', label, '->', err.code, '|', err.message.split('\n')[0]); }
}

check('equal uses Object.is', () => assert.equal(0.1 + 0.2, 0.3));
check('equal numbers', () => assert.equal(2 + 2, 4));
check('equal objects (reference!)', () => assert.equal({ a: 1 }, { a: 1 }));
check('deepEqual objects', () => assert.deepEqual({ a: 1, tags: ['x'] }, { a: 1, tags: ['x'] }));
check('deepEqual is type-strict', () => assert.deepEqual({ n: 1 }, { n: '1' }));
// ... ok([].length), match, throws with { name, message }, notDeepEqual
check('throws but nothing thrown', () => assert.throws(() => 42));
check('float with tolerance', () => assert.ok(Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON));

try {
  assert.deepEqual({ id: 7, items: ['pen', 'ink'] }, { id: 7, items: ['pen', 'paper'] });
} catch (err) {
  console.log('--- AssertionError message ---\n' + err.message);
}

await assert.rejects(Promise.reject(new Error('timeout')), /timeout/);
console.log('PASS rejects');
```

Output:

```
FAIL equal uses Object.is -> ERR_ASSERTION | Expected values to be strictly equal:
PASS equal numbers
FAIL equal objects (reference!) -> ERR_ASSERTION | Values have same structure but are not reference-
equal:
PASS deepEqual objects
FAIL deepEqual is type-strict -> ERR_ASSERTION | Expected values to be strictly deep-equal:
FAIL ok (truthy) -> ERR_ASSERTION | The expression evaluated to a falsy value:
PASS match regex
PASS throws with class + message
FAIL throws but nothing thrown -> ERR_ASSERTION | Missing expected exception.
PASS notDeepEqual
PASS float with tolerance
--- AssertionError message ---
Expected values to be strictly deep-equal:
+ actual - expected

  {
    id: 7,
    items: [
      'pen',
+     'ink'
-     'paper'
    ]
  }

PASS rejects
```

What to notice:

- `0.1 + 0.2` is not `0.3` in floating point (§2 Number) — compare floats with a tolerance, or keep money in integer cents (as the examples below do).
- `equal` on two separate objects fails even with identical contents; use `deepEqual` for structure.
- The diff (`+ actual - expected`) is what makes a failing test quick to diagnose — prefer one `deepEqual` on a whole object over many `equal`s on single fields.

TL;DR

Use `node:assert/strict`. `equal` for primitives/same reference, `deepEqual` for structures, `throws`/`await rejects` for errors, `match` for strings. A test fails when anything in it throws.

16.3 Writing and running tests with `node:test`

What it is

`node:test` is Node's built-in test runner (stable since Node 20). You import functions to *declare* tests; Node runs them and reports results. The `node:` prefix is mandatory for this module.

Function	Purpose
`test(name, [options], fn)` / alias `it`	One test. `fn` may be `async`; it receives a **test context** `t`
`describe(name, fn)` / alias `suite`	Group tests (can nest)
`before`/`after`, `beforeEach`/`afterEach`	Run once around the suite / around every test in it (and nested suites)
`t.test(name, fn)`	A **subtest** inside a test (`await` it)
`test.skip`, `test.todo`, `test.only` (+ `{ skip, todo, only, timeout }` options)	Skip; mark as not yet written (runs but failures don't count); run only these (with `--test-only`)
`t.mock`, `t.assert`, `t.plan(n)`, `t.signal`, `t.diagnostic(msg)`	Context helpers: auto-restoring mocks (§16.5), assertions incl. snapshots (§16.6), expected assertion count, an `AbortSignal` for cancellation, extra output

How it works

1. You run `node --test` (usually via `npm test`). With no arguments Node searches the current folder recursively (skipping `node_modules`) for files named like `*.test.{js,mjs,cjs}`, `*-test.*`, `*_test.*`, `test-*.*`, `test.*`, or anything inside a `test/` folder — and the same patterns with `.ts/.mts/.cts`.
2. Each file runs in **its own child process** by default (isolation: one file's globals or mocks can't leak into another; `--test-isolation=none` runs them in one process for speed).
3. Inside a file, tests run **in order**; `describe` callbacks run immediately to collect tests; hooks wrap them.
4. The **reporter** prints results: `spec` (default, the tree below), `tap`, `dot`, `junit` (CI dashboards), `lcov` (coverage); `--test-reporter-destination` writes to a file.
5. The **exit code** is 1 if any test failed — what CI checks.

Useful commands:

```
node --test                                # all matching files under the cwd
node --test test/cart.test.mjs             # specific files or globs
node --test --watch                        # re-run on file changes
node --test --test-name-pattern="Cart"     # only tests whose name matches a regex
node --test --test-reporter=dot            # compact output (--test-only: run only test.only)
node cart.test.mjs                         # a single file also runs as a plain script
```

Example — a unit test suite

The module under test, `code/src/cart.mjs`, is a shopping cart that stores prices in integer cents, plus `applyDiscount`, `formatPrice` and an async `checkout`:

```
export class Cart {
  #items = new Map();                                  // sku -> { sku, price, qty }
  add(sku, price, qty = 1) {
    if (typeof sku !== 'string' || sku === '') throw new TypeError('sku must be a non-empty string');
    if (!Number.isInteger(price) || price < 0) throw new RangeError('price must be integer cents >=
0');
    if (!Number.isInteger(qty) || qty < 1) throw new RangeError('qty must be an integer >= 1');
    // ... merge into the existing line for this sku; return this
  }
  // remove(sku), get count(), lines() (copies), subtotal()
}
export function applyDiscount(cents, percent) {
  if (percent < 0 || percent > 100) throw new RangeError(`bad percent: ${percent}`);
  return Math.round(cents * (100 - percent) / 100);
}
export function formatPrice(cents, currency = 'USD', locale = 'en-US') { /* Intl.NumberFormat */ }
export async function checkout(cart, { priceService, now = () => new Date() }) { /* §16.5 */ }
```

`code/cart.test.mjs`:

```
import { describe, it, test, before, beforeEach, afterEach, after } from 'node:test';
import assert from 'node:assert/strict';
import { Cart, applyDiscount, formatPrice } from './src/cart.mjs';

describe('Cart', () => {
  let cart;
  beforeEach(() => { cart = new Cart(); });          // fresh object for every test

  it('starts empty', () => {
    assert.equal(cart.count, 0);
    assert.equal(cart.subtotal(), 0);
  });

  it('adds items and merges the same sku', () => {
    // Arrange - Act - Assert
    cart.add('pen', 150, 2).add('ink', 900).add('pen', 150);
    assert.equal(cart.count, 4);
    assert.deepEqual(cart.lines(), [
      { sku: 'pen', price: 150, qty: 3 },
      { sku: 'ink', price: 900, qty: 1 },
    ]);
    assert.equal(cart.subtotal(), 1350);
  });
  // ... 'removes items'

  describe('validation', () => {
    const bad = [
      ['', 100, 1, TypeError],
      ['pen', -1, 1, RangeError],
      ['pen', 1.5, 1, RangeError],
      ['pen', 100, 0, RangeError],
    ];
    for (const [sku, price, qty, ErrorType] of bad) {     // table-driven tests
      it(`rejects add(${JSON.stringify(sku)}, ${price}, ${qty})`, () => {
        assert.throws(() => cart.add(sku, price, qty), ErrorType);
      });
    }
  });

  // ... 'lines() returns copies, not internal state'
});

describe('applyDiscount', () => {
  it('takes a percentage off and rounds to cents', () => {
    assert.equal(applyDiscount(1000, 10), 900);
    assert.equal(applyDiscount(999, 15), 849);         // 849.15 -> 849
    assert.equal(applyDiscount(1000, 0), 1000);
    assert.equal(applyDiscount(1000, 100), 0);
  });
  it('rejects percentages outside 0-100', () => {
    assert.throws(() => applyDiscount(1000, 101), { name: 'RangeError', message: 'bad percent: 101'
});
  });
});

test('formatPrice', async (t) => {                      // subtests with t.test
  await t.test('US dollars', () => assert.equal(formatPrice(123456), '$1,234.56'));
  await t.test('euros in Germany', () => assert.equal(formatPrice(123456, 'EUR', 'de-DE'),
'1.234,56 €'));
});

test.skip('multi-currency carts', () => {});           // reported as skipped
```

```
test.todo('coupon codes');                                  // reported as todo

describe('hooks order', () => {       // before/beforeEach/afterEach push to a log; after() prints it
  // ... it('one'), it('two')
});
```

Output of `node --test cart.test.mjs`:

```
# hooks ran: before > beforeEach > test1 > afterEach > beforeEach > test2 > afterEach
▶ Cart
  ✔ starts empty (0.425292ms)
  ✔ adds items and merges the same sku (0.421917ms)
  ✔ removes items (0.091708ms)
  ▶ validation
    ✔ rejects add("", 100, 1) (0.228958ms)
    ...
  ✔ validation (0.595709ms)
  ✔ lines() returns copies, not internal state (0.081625ms)
✔ Cart (2.060667ms)
...
﹣ multi-currency carts (0.020625ms) # SKIP
✔ coupon codes (0.016958ms) # TODO
...
ℹ tests 17
ℹ suites 4
ℹ pass 15
ℹ fail 0
...
ℹ skipped 1
ℹ todo 1
```

What each part shows:

1. **Arrange–Act–Assert (AAA)**: set up, do one action, check. One behaviour per test, named as a sentence so the report reads like a specification.
2. `beforeEach` **creates a fresh** `Cart`, so tests don't depend on each other's leftovers.
3. **Table-driven tests**: one named test per row of bad inputs — cheap to extend, each failure reported separately.
4. **Edge cases** are explicit: zero, 100 %, rounding (`849.15 → 849`), mutation of returned data.
5. The `'1.234,56 €'` expectation contains a **non-breaking space** before the euro sign — locale output is a classic source of surprising failures.
6. The `after` hook's `console.log` printed at the very top: each file runs in a child process whose stdout is printed before the formatted results. Use `t.diagnostic()` for messages inside the report.

Common mistakes

- Shared mutable state between tests (a module-level `cart`) — order-dependent, flaky.
- Testing implementation details (`#items`) instead of public behaviour — every refactor breaks tests.
- Leaving `test.only` or `{ only: true }` in committed code. Node runs *all* tests anyway unless `--test-only` is passed (it prints "'only' and 'runOnly' require the --test-only command-line option"), but Jest and Vitest really do skip the others — a whole suite can silently stop running.
- Many assertions checking unrelated behaviours in one test — the first failure hides the rest.
- Tests with no assertion at all (they always pass). `t.plan(n)` can enforce a count when callbacks are involved.

TL;DR

`import { test, describe, it, beforeEach } from 'node:test'`, run `node --test`. One behaviour per test, AAA, fresh fixtures in `beforeEach`, table-driven tests for many inputs, `skip`/`todo` for known gaps. Exit code 1 on failure.

16.4 Async tests and how tests fail

How async tests work

If a test returns a promise (any `async` function does), the runner **waits**: fulfilled = pass, rejected = fail. So `await` everything inside a test, especially `await assert.rejects(...)`. A test also fails if it exceeds its `timeout` option (default: none — set one for code that might hang).

Example — three ways to fail

`code/demo_failing.mjs` (skip-marked; run `node --test demo_failing.mjs`):

```
test('a wrong expectation', () => {
  assert.equal(applyDiscount(999, 15), 850);          // real answer is 849
});

test('a test that hangs', { timeout: 50 }, async () => {
  await new Promise((r) => setTimeout(r, 1000));
});

test('forgot to await the rejection', async () => {
  assert.rejects(Promise.resolve('no error'));          // missing await -> failure escapes the test
});
```

Output (stack trace shortened with `…`):

```
✖ a wrong expectation (0.612875ms)
✖ a test that hangs (51.228334ms)
✔ forgot to await the rejection (0.396875ms)
ℹ Error: Test "forgot to await the rejection" at demo_failing.mjs:14:1 generated asynchronous activity
after the test ended. This activity created the error "AssertionError [ERR_ASSERTION]: Missing
expected rejection." and would have caused the test to fail, but instead triggered an
unhandledRejection event.
ℹ tests 3
...
ℹ pass 1
ℹ fail 1
ℹ cancelled 1
...

✖ failing tests:

test at demo_failing.mjs:6:1
✖ a wrong expectation (0.612875ms)
  AssertionError [ERR_ASSERTION]: Expected values to be strictly equal:

  849 !== 850

      at TestContext.<anonymous> (file:///…/16-testing-and-typescript/code/demo_failing.mjs:7:10)
      …
    code: 'ERR_ASSERTION',
    actual: 849,
    expected: 850,
    ...
  }

test at demo_failing.mjs:10:1
✖ a test that hangs (51.228334ms)
  'test timed out after 50ms'
exit=1
```

How to read it:

1. **Wrong expectation** — actual vs expected plus the assertion's file:line. Decide whether the code or the test is wrong (here the test: 999 × 0.85 = 849.15).
2. **Timeout** — reported as *cancelled*; usually a promise that never settles or a server never closed.
3. **Missing `await`** — the dangerous one: marked **✔ passed** because the test finished before the assertion's promise rejected; Node only printed a warning that a busy CI log hides. *Every* `assert.rejects` and async call in a test gets `await` (`@typescript-eslint/no-floating-promises` catches this).

TL;DR

Make tests `async` and `await` everything; set `timeout` for code that could hang; a missing `await` can make a failing test look green.

16.5 Test doubles and mocking

What it is

A **test double** is a stand-in for a real dependency (network API, database, clock, random numbers) so the unit under test runs fast, deterministically, and offline. Terms you'll meet:

Double	What it is	Example
Dummy	Passed but never used	`{}` for an unused options argument
Stub	Returns canned answers	`getDiscount: async () => ({ percent: 10 })`
Spy	Records how it was called (and may call through)	`mock.fn()` → `.mock.calls`
Mock	A spy with expectations about calls, verified in the test	"called once with 500"
Fake	A working lightweight implementation	An in-memory repository instead of Postgres

In everyday speech people say "mock" for all of these.

Why it matters — and dependency injection

The easiest code to test takes its dependencies **as parameters** (**dependency injection**) instead of importing and calling them directly. `checkout` receives `priceService` and `now`:

```
export async function checkout(cart, { priceService, now = () => new Date() }) {
  if (cart.count === 0) throw new Error('cart is empty');
  const { percent } = await priceService.getDiscount(cart.subtotal());
  return { total: applyDiscount(cart.subtotal(), percent), at: now().toISOString() };
}
```

Production passes the real HTTP client and the real clock; tests pass a stub and a fixed clock — no mocking library needed.

The `node:test` mocking API

API	Does
`mock.fn(impl?)`	A spy function; `fn.mock.calls` (each `{ arguments, result, error, this }`), `fn.mock.callCount()`
`fn.mock.mockImplementation(f)` / `mockImplementationOnce(f)`	Change behaviour (for all / the next call)
`mock.method(obj, 'name', impl?)`	Replace a method on an existing object, keep a spy on it
`mock.getter` / `setter` / `property`	Replace accessors / a property value
`t.mock.*`	Same, but **auto-restored when the test ends** — prefer it over the global `mock` (which needs `mock.restoreAll()`)
`t.mock.timers.enable({ apis: ['setTimeout', 'Date', …], now })` + `tick(ms)`	**Fake timers** — control `setTimeout`/`setInterval`/`setImmediate`/`Date` without waiting
`t.mock.module(specifier, { namedExports, defaultExport })`	Replace what `import` returns (needs `--experimental-test-module-mocks`)

`code/checkout.test.mjs`:

```
import { describe, it, mock } from 'node:test';
import assert from 'node:assert/strict';
import { Cart, checkout } from './src/cart.mjs';

const fixedNow = () => new Date('2026-01-15T10:00:00Z');   // an injected, fixed clock

describe('checkout (async + dependency injection)', () => {
  it('applies the discount returned by the price service (fake object)', async () => {
    const fakePriceService = { getDiscount: async () => ({ percent: 10 }) };
    const cart = new Cart().add('pen', 1000);
    const receipt = await checkout(cart, { priceService: fakePriceService, now: fixedNow });
    assert.deepEqual(receipt, { total: 900, at: '2026-01-15T10:00:00.000Z' });
  });
  // ... 'rejects an empty cart', 'propagates service failures' (await assert.rejects)
  it('calls the service once with the subtotal (mock.fn spy)', async () => {
    const getDiscount = mock.fn(async () => ({ percent: 0 }));
    await checkout(new Cart().add('ink', 250, 2), { priceService: { getDiscount }, now: fixedNow });
    assert.equal(getDiscount.mock.callCount(), 1);
    assert.deepEqual(getDiscount.mock.calls[0].arguments, [500]);
  });
});

describe('mock.method and t.mock', () => {
  const api = { async fetchUser(id) { throw new Error('real network call - must not run in tests'); }
};
  it('replaces a method for one test; t.mock restores it automatically', async (t) => {
    const m = t.mock.method(api, 'fetchUser', async (id) => ({ id, name: 'Test User' }));
    assert.deepEqual(await api.fetchUser(7), { id: 7, name: 'Test User' });
    assert.equal(m.mock.callCount(), 1);
  });
  // ... 'after the previous test the real method is back' (assert.rejects on the real one)
  it('mockImplementationOnce: fail first, then succeed (retry logic)', async (t) => {
    const flaky = t.mock.fn(async () => 'ok');
    flaky.mock.mockImplementationOnce(async () => { throw new Error('ECONNRESET'); });
    assert.equal(await withRetry(flaky), 'ok');      // withRetry: try up to 3 times
    assert.equal(flaky.mock.callCount(), 2);
  });
});

describe('fake timers', () => {
  function debounce(fn, ms) {
    let id;
    return (...args) => { clearTimeout(id); id = setTimeout(() => fn(...args), ms); };
  }
  it('debounce calls once, after the quiet period - without really waiting', (t) => {
    t.mock.timers.enable({ apis: ['setTimeout'] });
    const save = t.mock.fn();
    const debouncedSave = debounce(save, 300);
    debouncedSave('a'); debouncedSave('ab'); debouncedSave('abc');
    t.mock.timers.tick(299);
    assert.equal(save.mock.callCount(), 0);
    t.mock.timers.tick(1);
    assert.equal(save.mock.callCount(), 1);
    assert.deepEqual(save.mock.calls[0].arguments, ['abc']);
  });
  // ... 'Date can be frozen too': enable({ apis: ['Date'], now: new Date('2030-05-01T00:00:00Z') })
});
```

Output of `node --test checkout.test.mjs`:

```
▶ checkout (async + dependency injection)
  ✔ applies the discount returned by the price service (fake object) (1.343458ms)
  ...
✔ checkout (async + dependency injection) (2.544709ms)
▶ mock.method and t.mock
  ...
▶ fake timers
  ✔ debounce calls once, after the quiet period - without really waiting (0.290834ms)
  ✔ Date can be frozen too (0.12175ms)
✔ fake timers (0.520417ms)
ℹ tests 9
ℹ suites 3
ℹ pass 9
ℹ fail 0
...
```

The debounce test checks a 300 ms delay in under half a millisecond: fake timers move time only when you call `tick`. (Debounce itself is built in Chapter 17.)

Module mocking

When code imports a dependency directly (no injection), you can replace the **module** itself. `code/module_mock_demo.mjs` mocks the built-in `node:os` that `src/greeting.mjs` imports:

```
test('greeting uses the mocked os module', async (t) => {
  t.mock.module('node:os', { namedExports: { userInfo: () => ({ username: 'ana' }) } });
  const { greeting } = await import('./src/greeting.mjs');   // import AFTER mocking
  assert.equal(greeting(), 'Hello, ana!');
});
```

Output of `node --test --experimental-test-module-mocks module_mock_demo.mjs`:

```
(node:77997) ExperimentalWarning: Module mocking is an experimental feature and might change at any
time
(Use `node --trace-warnings ...` to show where the warning was created)
✔ greeting uses the mocked os module (2.704792ms)
ℹ tests 1
...
ℹ pass 1
ℹ fail 0
...
```

Module mocking is still **experimental** in Node 24 (hence the flag and warning), and the module must be imported *after* the mock is set up — hence the dynamic `import()` (Vitest's `vi.mock` and Jest's `jest.mock` hoist mocks above imports for you). Prefer dependency injection; use module mocks when you can't change the code.

When not to mock

- **Don't mock what you own and can run cheaply** (pure functions, your own small modules) — use the real thing.
- **Don't mock the unit under test** — you would be testing the mock.
- Too many mocks give tests that pass while production fails (doubles drift from reality) — balance with integration tests against real (containerized) databases and **contract tests** for external APIs.
- For HTTP, prefer a real local server (§15.10's port-0 pattern) or an interception library (MSW, undici `MockAgent`) over hand-mocking `fetch`.

> **In the interview**

Q: Stub vs mock vs fake? "A stub returns canned data; a mock (or spy) records calls so you can assert on them; a fake is a simple working implementation, like an in-memory DB. I prefer injecting dependencies so tests can pass fakes without a mocking framework."

- *How do you test code using `setTimeout` or `Date.now()`?* Fake timers (`t.mock.timers`), or inject a clock function.
- *What makes tests flaky?* Real time, real network, shared state between tests, test order, unawaited promises, locale/timezone differences.

> **TL;DR**

Inject dependencies and pass stubs/fakes; use `t.mock.fn` / `t.mock.method` for spies (auto-restored); fake timers for time; module mocks as a last resort. Don't over-mock.

16.6 Snapshot tests and code coverage

Snapshot testing

A **snapshot test** saves a function's output the first time and fails later if it changes — good for large, stable outputs (a receipt, a CLI help screen) that are tedious to write by hand. In Node, `t.assert.snapshot(value)` is **stable since Node 23.4**.

`code/snapshot.test.mjs`:

```
function receiptText(cart) {
  const rows = cart.lines().map((l) => `${l.sku.padEnd(6)} x${l.qty}  ${(l.price * l.qty /
100).toFixed(2).padStart(7)}`);
  return [...rows, '-'.repeat(20), `TOTAL ${(cart.subtotal() /
100).toFixed(2).padStart(13)}`].join('\n');
}
test('receipt layout', (t) => {
  const cart = new Cart().add('pen', 150, 3).add('ink', 900);
  t.assert.snapshot(receiptText(cart));
});
```

Workflow:

1. First run: `node --test --test-update-snapshots snapshot.test.mjs` writes `snapshot.test.mjs.snapshot`:

   ```
   exports[`receipt layout 1`] = `
   "pen    x3     4.50\\nink    x1     9.00\\n--------------------\\nTOTAL         13.50"
   `;
   ```

2. Commit the `.snapshot` file. Later runs compare against it; without the file, the test fails with "Missing snapshots can be generated by rerunning the command with the --test-update-snapshots flag".
3. When the output changes **on purpose**, rerun with `--test-update-snapshots` and review the diff in code review.

The danger: updating snapshots without reading them. Keep them small and review them like code.

Code coverage

Coverage measures which lines, branches and functions your tests executed. Output of `node --test --experimental-test-coverage` in `code/` (all test files):

```
ℹ start of coverage report
ℹ ---------------------------------------------------------
ℹ file      | line % | branch % | funcs % | uncovered lines
ℹ ---------------------------------------------------------
ℹ src       |        |          |         |
ℹ  cart.mjs | 100.00 |   100.00 |   90.91 |
ℹ  money.ts | 100.00 |    88.89 |  100.00 |
ℹ ---------------------------------------------------------
ℹ all files | 100.00 |    97.14 |   93.33 |
ℹ ---------------------------------------------------------
ℹ end of coverage report
```

(Test files are excluded by default.) `cart.mjs` shows 90.91 % of functions: the default clock `now = () => new Date()` never ran, because every test injected its own — coverage points at code no test exercises. Flags: `--test-coverage-include`/`-exclude`, `--test-coverage-lines=80` (fail below a threshold), `--test-reporter=lcov` for CI services. The flag still says "experimental" in Node 24.

Coverage is a **floor, not a goal**: 100 % means every line *ran*, not that results were *checked* — use it to find untested areas, don't chase the number.

> **TL;DR**
>
> Snapshots record output and flag changes — commit and review them. Coverage (`--experimental-test-coverage`) shows unexecuted code; treat it as a hint, not a target.

16.7 Good tests, and other runners

What makes a good test (F.I.R.S.T. and more)

Property	Meaning
Fast	Milliseconds; slow suites stop being run
Independent/Isolated	No shared state, any order
Repeatable	Same result every run: fixed clock, seeded randomness, no real network
Self-validating	Pass/fail by assertions, not by a human reading logs
Timely	Written with (or before — **TDD**, test-driven development: red → green → refactor) the code
Behaviour-focused	Tests public behaviour; survives refactoring
Readable	Named as sentences; AAA structure; a failure message says what broke

Same ideas in Vitest and Jest

Concept	node:test	Vitest	Jest
Import	`from 'node:test'` + `node:assert/strict`	`from 'vitest'` (or globals)	globals
Test / group	`test` / `it`, `describe`	same	same
Assert equal	`assert.equal(a, b)`	`expect(a).toBe(b)`	same
Deep equal	`assert.deepEqual`	`expect(a).toEqual(b)` / `toStrictEqual`	same
Throws	`assert.throws(fn, Err)`	`expect(fn).toThrow(Err)`	same
Rejects	`await assert.rejects(p)`	`await expect(p).rejects.toThrow()`	same
Spy	`t.mock.fn()`	`vi.fn()`	`jest.fn()`
Method mock	`t.mock.method(o, 'm')`	`vi.spyOn(o, 'm')`	`jest.spyOn`
Fake timers	`t.mock.timers.enable()` / `tick`	`vi.useFakeTimers()` / `vi.advanceTimersByTime`	`jest.useFakeTimers()`
Module mock	`t.mock.module` (flag)	`vi.mock()` (hoisted)	`jest.mock()` (hoisted)
Snapshot	`t.assert.snapshot`	`toMatchSnapshot()`	same
Run	`node --test`	`vitest`	`jest`

> **TL;DR**

Fast, isolated, repeatable, behaviour-focused, readable tests. The API names change between runners; the concepts don't.

16.8 TypeScript: what it is and how it's checked

What it is

TypeScript (TS) is JavaScript plus a **static type system**, developed by Microsoft since 2012. You add **type annotations** (`let pages: number`), and the **type checker** (the compiler `tsc`, or your editor using it) verifies that values are used consistently — *before* the code runs. The types are then **erased**: what executes is ordinary JavaScript. TS has **no runtime** of its own.

Key terms:

- **Static typing**: types are checked by analyzing code, not by running it (JavaScript itself is dynamically typed — §2).
- **Type inference**: TS works out many types by itself (`let pages = 30` is `number`), so you annotate mainly function parameters and public APIs.
- **Structural typing**: compatibility is decided by **shape**, not by name. Any object with `{ id: number; name: string }` is a valid `User`, whether or not it was declared as one ("duck typing" checked at compile time).
- **Gradual typing**: you can adopt TS file by file; `any` switches checking off for a value.

- **Soundness**: TS is deliberately not fully sound — `any`, assertions and some index operations let wrong types through. Types are strong evidence, not proof.

Analogy: a spell-checker for program structure. It underlines mistakes as you type; the printed letter (the JavaScript) contains no red lines.

Why it matters

Catches typos, wrong argument orders, missing `null` checks and forgotten cases at edit time; powers accurate autocomplete and safe renames; documents APIs. It is the industry default for serious front-end and Node projects (React, Angular, Vue and Next.js all support it first-class).

Trade-offs

Benefit	Cost
Bugs caught before running; editor support	A check step and a `tsconfig.json`; a learning curve for advanced types
Self-documenting APIs	Types lie if you misuse `any` / `as`, and never validate **runtime** data (API JSON still needs a check or a schema library such as Zod)

`tsc` and TypeScript 7

`tsc` is the TypeScript compiler. Its two jobs are **type-checking** and **emitting** JavaScript. Today, emitting is often left to faster tools (esbuild, Vite, SWC, or Node's type stripping, §16.9) and `tsc` is run only for checking: **`tsc --noEmit`**.

TypeScript 7.0 (released 8 July 2026) is a **native port of the compiler written in Go**; Microsoft reports full builds roughly 8–12× faster than TypeScript 6, with the same `typescript` npm package and `tsc` command. It also changes defaults and removes legacy options:

In TypeScript 7	Details
New defaults	`strict: true`, `module: esnext`, `target`: the current stable ECMAScript version, `types: []` (you list `@types` packages explicitly), `noUncheckedSideEffectImports: true`
Removed	`target: es5`, `downlevelIteration`, `moduleResolution: node` / `node10` / `classic`, `module: amd/umd/systemjs/none`, `baseUrl` (use `paths`)
Not yet	A stable programmatic (JavaScript) API — expected in 7.1; tools that call the compiler's API may lag

Everyday commands (not run — the project in `code/` already has TypeScript available):

```
npm install -D typescript @types/node   # compiler + Node's type definitions
npx tsc --init                          # create a tsconfig.json
npx tsc --noEmit                        # type-check the project (what CI runs)
npx tsc --noEmit --watch                # re-check on every save
```

`tsconfig.json`

The project's `code/tsconfig.json`:

```jsonc
{
  "compilerOptions": {
    "target": "es2024",
    "lib": ["es2024", "dom"],
    "module": "nodenext",
    "strict": true,
    "noEmit": true,
    "noUncheckedIndexedAccess": true,
    "allowImportingTsExtensions": true,
    "erasableSyntaxOnly": true,
    "verbatimModuleSyntax": true,
    "allowJs": true,
    "checkJs": false
  },
  "include": ["*.ts", "src/*.ts", "15_jsdoc.mjs"],
  "exclude": ["ts_errors.ts", "no_typecheck_demo.ts", "*.test.ts"]
}
```

Option	Why
`target`, `lib`	Output syntax and available built-in APIs (`dom` provides `console`, as `@types/node` isn't installed)
`module: nodenext`	Follow Node's real module rules (reads `"type"` from `package.json`)
`strict`	`strictNullChecks`, `noImplicitAny`, … — always on (default in TS 7)
`noEmit`	Check only; something else runs the code
`noUncheckedIndexedAccess`	`arr[i]` and `record[key]` are `T | undefined` — catches out-of-range reads
`allowImportingTsExtensions`	Allow `import './shapes.ts'` (Node requires the real extension)
`erasableSyntaxOnly`	Reject TS features that need code generation (enum, namespaces with code, parameter properties) — exactly what Node can't strip (TS 5.8+)
`verbatimModuleSyntax`	Force `import type` for type-only imports, so removing types never changes which modules load

Running the checker on the whole project:

```
$ tsc --noEmit
$ echo $?
0
```

No output and exit code 0 means no type errors.

TL;DR

TypeScript = JavaScript + static types that are checked by `tsc` and erased before running. Use `strict`, check with `tsc --noEmit`. TypeScript 7 is the Go-native, ~10× faster compiler with stricter defaults.

16.9 Running TypeScript: Node's type stripping and other tools

What it is

Since **Node 23.6** (and backported to 22.18), `node file.ts` works with no flags: Node **strips the type annotations** — replacing them with whitespace, so line and column numbers stay correct without source maps — and runs the remaining JavaScript. The feature stopped printing an experimental warning in Node 24.3 and was declared **stable in Node 24.12**.

Stripping is not checking. `code/no_typecheck_demo.ts`:

```
const price: number = 5;
const qty: number = '2';                  // type error: string is not assignable to number
const total: number = price + qty;        // JS concatenates: 5 + '2' -> '52'
console.log('total =', JSON.stringify(total), '| typeof', typeof total);
```

Output of `node no_typecheck_demo.ts`, then `tsc` on the same file:

```
total = "52" | typeof string
no_typecheck_demo.ts(4,7): error TS2322: Type 'string' is not assignable to type 'number'.
```

Node ran code whose `number` annotation holds the string `"52"`. **Always run** `tsc --noEmit` (editor, pre-commit, CI) as well.

The rules for Node's type stripping

1. **Only erasable syntax.** Things that generate JavaScript are rejected with `ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX`: `enum`, `namespace`/`module` blocks containing code, **parameter properties** (`constructor(public x: number)`), `import x = require()` aliases, and legacy decorators. (`--experimental-transform-types` enables them, with a warning.) Turn on `erasableSyntaxOnly` so the editor flags them.
2. **Write real file extensions** in relative imports: `import { area } from './shapes.ts'`.
3. **Use** `import type` (or inline `type` modifiers) for imports that are only types (`verbatimModuleSyntax` enforces it).
4. `tsconfig.json` **is ignored** by Node — no `paths` aliases, no `target` downlevelling.
5. **Not in** `node_modules`: Node refuses to strip TypeScript files inside dependencies; packages must publish JavaScript (+ `.d.ts` type declarations).
6. `.ts` follows the same ESM/CommonJS rules as `.js`; `.mts`/`.cts` force one.

A first draft of `11_narrowing.ts` used a parameter property and hit rule 1:

```
class HttpError extends Error { constructor(public status: number) { super(`HTTP ${status}`); } }
                                                    ^^^^^^^^^^^^^^

SyntaxError [ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX]: TypeScript parameter property is not supported in
strip-only mode
```

`tsc` with `erasableSyntaxOnly` reports it as `TS1294`. Fix: declare the field and assign it in the constructor.

`code/14_node_runs_ts.ts` follows all the rules:

```ts
import type { Shape } from './shapes.ts';
import { area } from './shapes.ts';

const shapes: Shape[] = [
  { kind: 'rect', w: 2, h: 5 },
  { kind: 'circle', r: 1 },
];
const total: number = shapes.reduce((sum, s) => sum + area(s), 0);
console.log('total area:', total.toFixed(3));
console.log('shapes:', shapes.map((s): string => s.kind).join(', '));
```

Output of `node 14_node_runs_ts.ts`:

```
total area: 13.142
shapes: rect, circle
```

Other ways to run TypeScript

Tool	What it does	Type-checks?
`node file.ts`	Strips erasable types	No
`tsx` (npm)	esbuild-powered runner; supports all TS syntax and `tsconfig` paths	No
`esbuild` / `Vite` / `SWC`	Transpile or bundle TS for browsers and Node (this book's runner uses esbuild)	No
`tsc`	Checks, and can emit `.js` + `.d.ts` (needed when publishing libraries)	**Yes**
`Deno` / `Bun`	Run TS natively (§15.12); `deno check` type-checks	Deno: on request

Typical setup: *run* with Node/tsx/Vite, *check* with `tsc --noEmit`, *publish libraries* with `tsc` emitting declarations.

TL;DR

`node app.ts` works in Node 24 (stable type stripping) but never type-checks. Use only erasable syntax, `.ts` extensions in imports, `import type`, and keep `tsc --noEmit` in CI.

16.10 Everyday types

How it works

`code/10_ts_basics.ts` walks through the everyday types (key parts):

```ts
// 1) Primitive annotations - and inference
let title: string = 'Testing & TypeScript';
let pages = 30;                          // inferred: number
const lang = 'ts';                       // inferred: the LITERAL type "ts" (const)
// ... also bigint and null annotations

// 2) Arrays and tuples
const scores: number[] = [90, 72, 85];
const point: [x: number, y: number] = [3, 4];          // tuple: fixed length & types

// 3) Object types: type alias vs interface; optional and readonly properties
type User = {
  readonly id: number;
  name: string;
  email?: string;                        // optional: string | undefined
};
interface Product { sku: string; price: number }
interface Product { tags: string[] }     // interfaces MERGE (declaration merging)

// 4) Functions: parameter & return types, default params, void
function greet(user: User, greeting = 'Hello'): string { return `${greeting}, ${user.name}!`; }

// 5) Union and literal types: type Status = 'draft' | 'published' | 'archived'; let id: number |
string

// 6) any vs unknown: unknown forces you to check before use
function parse(json: string): unknown { return JSON.parse(json); }
const data = parse('{"count": 3}');
if (typeof data === 'object' && data !== null && 'count' in data && typeof data.count === 'number') {
  console.log('6) count + 1 =', data.count + 1);
}

// 7) Type assertions and the non-null assertion - you override the checker (use sparingly)
const maybe = scores.find((n) => n > 80);           // number | undefined
console.log('7)', (maybe as number).toFixed(1), maybe!.toFixed(0));
// 8) ... types disappear at runtime
```

Output of `node 10_ts_basics.ts`:

```
1) string number ts 1000000000000000000n null
2) 3 Ana+Bo 5
3) (no email) office
4) Hello, Ana!
4) Hi, Bo!
5) [LIVE] [DRAFT] A-42
6) count + 1 = 4
7) 90.0 90
8) typeof User at runtime: undefined
```

The type vocabulary

Type	Meaning
`string`, `number`, `boolean`, `bigint`, `symbol`, `null`, `undefined`	The JS primitives (lower-case; `String` is the wrapper object)
`'draft'`, `42`, `true`	**Literal types** — exactly that value
`A | B`	**Union** — either
`A & B`	**Intersection** — both at once (combine object types)
`T[]` / `Array<T>`	Array
`[string, number]`	**Tuple** — fixed length, typed positions
`{ a: string; b?: number }`	Object type; `?` optional, `readonly` prevents reassignment
`(x: number) => string`	Function type
`any`	Opt out of checking — anything goes, errors spread silently
`unknown`	"Could be anything" — you must narrow it before use. **Use for untrusted input** (JSON, `catch (e)`)
`never`	No value possible (a function that always throws; the impossible branch of a switch)
`void`	A function returns nothing useful
`object`	Any non-primitive

`type` **vs** `interface`: both describe object shapes. `interface` can be **reopened** (declaration merging, used to extend library types) and `extends` others; `type` can also name unions, tuples and computed types. Many teams use `interface` for object shapes and `type` for the rest — consistency matters more than the choice.

Seeing the checker work

`code/ts_errors.ts` (skip-marked) contains nine deliberate mistakes:

```ts
type User = { id: number; name: string; email?: string };

const u: User = { id: 1, name: 'Ana', age: 30 };            // 1) excess property
u.id = '2';                                                 // 2) wrong type
console.log(u.email.toLowerCase());                         // 3) possibly undefined

function total(prices: number[]): number {
  return prices.reduce((a, b) => a + b, '');                // 4) string is not number
}
total(['1.50', '2.00']);                                    // 5) string[] is not number[]

type Status = 'draft' | 'published';
const s: Status = 'publised';                               // 6) typo caught

function len(x: string | number) {
  return x.length;                                          // 7) number has no length
}

class Point { constructor(private x: number) {} }           // 8) not erasable (Node can't strip)
enum Color { Red, Green }                                   // 9) not erasable either
```

Output of `tsc --ignoreConfig --noEmit --strict --target es2024 --erasableSyntaxOnly ts_errors.ts`:

```
ts_errors.ts(4,39): error TS2353: Object literal may only specify known properties, and 'age' does not
exist in type 'User'.
ts_errors.ts(5,1): error TS2322: Type 'string' is not assignable to type 'number'.
ts_errors.ts(6,13): error TS18048: 'u.email' is possibly 'undefined'.
ts_errors.ts(9,3): error TS2322: Type 'string' is not assignable to type 'number'.
...
ts_errors.ts(14,7): error TS2820: Type '"publised"' is not assignable to type 'Status'. Did you mean
'"published"'?
ts_errors.ts(17,12): error TS2339: Property 'length' does not exist on type 'string | number'.
  Property 'length' does not exist on type 'number'.
ts_errors.ts(20,27): error TS1294: This syntax is not allowed when 'erasableSyntaxOnly' is enabled.
...
```

Each line is `file(line,column): error TSxxxx: message` (`--ignoreConfig` is needed because TypeScript 7 won't combine a present `tsconfig.json` with file names on the command line). Error 3 is the famous one: with `strict`, "possibly `undefined`" catches `TypeError: Cannot read properties of undefined` (§11.10) before it happens. Error 1, the **excess property check**, applies only to object literals written where a type is expected; an object stored in a variable first may carry extra properties (structural typing).

Common mistakes

- `any` to silence an error — it spreads to everything derived from it. Use `unknown` and narrow.
- Overusing `as` and `!` — when you're wrong, the runtime error comes back.
- Annotating everything instead of letting inference handle locals.
- Trusting types for external data: `(await res.json()) as User` checks nothing. Validate at the boundary (schema library or a hand-written guard).

TL;DR

Annotate parameters and public APIs; let inference do the rest. Unions and literal types model real states; `unknown` for untrusted data, avoid `any`; `strict` catches `undefined` bugs.

16.11 Narrowing

What it is

Narrowing is how TS refines a broad type (a union, `unknown`) to a more specific one inside a branch, based on the checks you write. The checks are ordinary JavaScript — TS just understands what they prove.

Check	Narrows by
`typeof x === 'string'`	Primitive type
`x === null`, `x !== undefined`, truthiness `if (x)`	Removing `null`/`undefined` (careful: truthiness also removes `0` and `''`)
`x instanceof Date`	Class
`'code' in x`	Property existence
`s.kind === 'circle'`	**Discriminant** property of a union
`function isCat(p): p is Cat`	Your own **type predicate**
`function assertX(v): asserts v is X`	**Assertion function** — narrows everything after the call

Discriminated unions and exhaustiveness

A **discriminated (tagged) union** is a union of object types that share one literal property (the **discriminant**, here `kind`). Checking it tells TS exactly which member you have. Assigning the leftover to `never` in `default` makes the compiler **force you to handle every case**: add `{ kind: 'hexagon' }` to `Shape` and `area` stops compiling until you add a branch.

`code/11_narrowing.ts` (key parts):

```ts
type Shape =
  | { kind: 'circle'; radius: number }
  | { kind: 'rect'; width: number; height: number }
  | { kind: 'triangle'; base: number; height: number };

function area(s: Shape): number {
  switch (s.kind) {
    case 'circle': return Math.PI * s.radius ** 2;
    case 'rect': return s.width * s.height;
    case 'triangle': return (s.base * s.height) / 2;
    default: {
      const unreachable: never = s;        // compile error here if a new kind is not handled
      throw new Error(`unknown shape ${JSON.stringify(unreachable)}`);
    }
  }
}

type Result<T> = { ok: true; value: T } | { ok: false; error: string };   // parsePort(text): Result<number>

function isCat(pet: Cat | Dog): pet is Cat { return 'meow' in pet; }   // type predicate

function assertIsString(v: unknown, name: string): asserts v is string {
  if (typeof v !== 'string') throw new TypeError(`${name} must be a string`);
}
```

Output of `node 11_narrowing.ts`:

```
1) text of length 3 | number 3.14 | nothing
2) http 404 | code ENOENT | error x
3) circle=3.14, rect=6.00, triangle=10.00
4) port 8080
4) bad port "http"
5) meow woof
5) after assert: HELLO
6) [ 'Pune', 'unknown' ]
```

Real-world examples

Redux actions (`{ type: 'added' } | { type: 'removed' }`, React Handbook ch6), API responses (`{ status: 'loading' } | { status: 'success'; data } | { status: 'error'; error }`), and `Result` types in libraries that avoid exceptions are all discriminated unions.

In the interview

Q: How do you make sure every case of a union is handled? "Model it as a discriminated union, switch on the discriminant, and assign the value to a `never`-typed variable in `default`. Adding a new member makes that assignment a compile error."

- *What is a type predicate?* A function returning `x is T`; when it returns true, TS narrows the argument to `T` in the caller.

Self-check

1. After `if (typeof x === 'string') return;`, what is `x` if its type was `string | number`? → `number`.
2. Why prefer `x !== undefined` over `if (x)` for a `number | undefined`? → Truthiness would also drop `0`.
3. What does `asserts v is string` do? → After the call returns, `v` is treated as `string`; the function must throw otherwise.

TL;DR

TS narrows through `typeof`, `instanceof`, `in`, equality, and discriminants. Model states as discriminated unions; use `never` for exhaustive switches; write type predicates and assertion functions for custom checks.

16.12 Generics and utility types

What it is

A **generic** is a type parameter — a placeholder (`T`) filled in when the function or type is used. It lets one function work for many types **without losing** the specific type (as `any` would).

Analogy: a labelled storage box. A `Box<Book>` and a `Box<Plate>` are built the same way, but the label tells you what comes out.

`code/12_generics.ts` (key parts):

```ts
function first<T>(items: readonly T[]): T | undefined { return items[0]; }
const n = first([10, 20]);          // number | undefined  (T inferred)

function longest<T extends { length: number }>(a: T, b: T): T { /* ... */ }   // constraint

function pluck<T, K extends keyof T>(rows: T[], key: K): T[K][] {   // keyof + indexed access
  return rows.map((r) => r[key]);
}
pluck(users, 'name');               // string[]  -  pluck(users, 'nme') would not compile

class TypedCache<K, V> { /* set(k: K, v: V): this; get(k: K): V | undefined - wraps a Map<K, V> */ }

interface Todo { id: number; title: string; done: boolean; tags: string[] }
type TodoPatch = Partial<Omit<Todo, 'id'>>;
type ById = Record<number, Pick<Todo, 'id' | 'title'>>;

type Flags<T> = { [K in keyof T]: boolean };              // mapped type
type ElementOf<T> = T extends readonly (infer E)[] ? E : never;   // conditional type
```

Output of `node 12_generics.ts`:

```
1) 10 a
2) apple [ 1, 2, 3 ]
3) [ 'Ana', 'Bo' ] [ true, false ]
4) 2 3
5) true write ch16
5) [ 1, 'x' ] true
6) { title: true, done: false } urgent
```

Built-in utility types

Utility	Result
`Partial<T>` / `Required<T>`	All properties optional / required
`Readonly<T>`	All properties `readonly` (shallow)
`Pick<T, K>` / `Omit<T, K>`	Keep / drop some properties
`Record<K, V>`	Object with keys `K` and values `V`
`Exclude<U, X>` / `Extract<U, X>`	Remove / keep union members
`NonNullable<T>`	Remove `null` and `undefined`
`ReturnType<F>` / `Parameters<F>`	A function's return type / parameter tuple
`Awaited<T>`	The value a promise resolves to

Mapped types (`{ [K in keyof T]: … }`) transform every property; **conditional types** (`T extends X ? A : B`, with `infer`) pick types from other types. Utility types are built from them; application code rarely needs them.

Common mistakes

- Generic parameters that appear only once (`function f<T>(x: T): void`) — then `unknown` is simpler.
- `any[]` instead of `T[]` — loses the element type.
- Deep type gymnastics in application code — hard to read; keep types boring.

TL;DR

Generics (`<T>`) keep precise types across reusable code; constrain with `extends`, use `keyof`/`T[K]` for property-safe helpers. Know `Partial`, `Pick`, `Omit`, `Record`, `ReturnType`, `Awaited`.

16.13 Classes, `as const`, `satisfies`, and enums

`code/13_classes_and_modern_ts.ts` (key parts):

```ts
interface Account { deposit(cents: number): void; readonly balance: number }

class BankAccount implements Account {
  #balance = 0;                              // JS private field (runtime)
  private readonly history: number[] = [];    // TS private (compile-time only)
  protected owner: string;
  // ... constructor(owner), deposit(cents), get balance()
}

const ROLES = ['viewer', 'editor', 'admin'] as const;
type Role = (typeof ROLES)[number];           // 'viewer' | 'editor' | 'admin'

const LogLevel = { Debug: 0, Info: 1, Warn: 2, Error: 3 } as const;   // enum alternative

type Route = { path: string; auth: boolean };
const routes = {
  home: { path: '/', auth: false },
  admin: { path: '/admin', auth: true },
} satisfies Record<string, Route>;
// stock['ink'] on a Record<string, number> is number | undefined (noUncheckedIndexedAccess)
```

Output of `node 13_classes_and_modern_ts.ts`:

```
1) Ana's savings: 5000 | accounts: 1
2) viewer:false editor:true admin:true
3) false true
4) home,admin /admin
5) 0
```

What each feature is for:

Feature	Meaning	Note
`implements Account`	The class must match the interface's shape	Checked at compile time only
`private` / `protected` / `public`	Visibility **checked by TS only** — at runtime the property is ordinary	Use JS `#private` (§9) when privacy must hold at runtime
`readonly`	Can't be reassigned after construction	Compile time only
`abstract class`	Can't be instantiated; subclasses implement abstract members	Erasable
`as const`	Make a literal deeply `readonly` with literal types	Derive unions from data (`typeof ROLES[number]`) — one source of truth
`satisfies T` (TS 4.9+)	Check a value against `T` **without widening** it	`routes.admin` stays known; with `: Record<string, Route>` it would be "any string key"
`enum`	Named constants that generate a runtime object	**Not erasable** — rejected by Node's type stripping. Prefer a union of literals or an `as const` object

> **TL;DR**
>
> TS classes add `implements`, `private/protected/readonly` (compile-time) on top of JS classes; use `#private` for runtime privacy. Prefer `as const` objects/unions to `enum`; use `satisfies` to validate config objects while keeping precise types.

16.14 Tests written in TypeScript

`node --test` also finds `*.test.ts` files and type-strips them, so TypeScript projects can use the built-in runner with **no build step**. The module `code/src/money.ts` splits money without losing cents:

```ts
export type Currency = 'USD' | 'EUR' | 'INR';
export interface Money { readonly cents: number; readonly currency: Currency }
export function money(cents: number, currency: Currency = 'USD'): Money { /* validate, freeze */ }
export function add(a: Money, b: Money): Money { /* same currency only */ }
export function split(m: Money, n: number): Money[] {
  const base = Math.floor(m.cents / n);
  const extra = m.cents % n;
  return Array.from({ length: n }, (_, i) => money(base + (i < extra ? 1 : 0), m.currency));
}
```

`code/money.test.ts`:

```ts
import { describe, it } from 'node:test';
import assert from 'node:assert/strict';
import { money, add, split, type Money } from './src/money.ts';

describe('money', () => {
  it('adds amounts in the same currency', () => {
    assert.deepEqual(add(money(150), money(275)), { cents: 425, currency: 'USD' });
  });
  // ... 'refuses to mix currencies'
  it('splits without losing a cent', () => {
    const parts: Money[] = split(money(1000), 3);
    assert.deepEqual(parts.map((p) => p.cents), [334, 333, 333]);
    assert.equal(parts.reduce((t, p) => t + p.cents, 0), 1000);
  });
  it('values are immutable', () => {
    const m = money(5);
    assert.throws(() => { (m as { cents: number }).cents = 6; }, TypeError);
  });
});
```

Output of `node --test money.test.ts`:

```
▶ money
  ✔ adds amounts in the same currency (0.494792ms)
  ...
  ✔ values are immutable (0.055667ms)
✔ money (1.196625ms)
ℹ tests 4
...
ℹ pass 4
ℹ fail 0
...
```

"values are immutable" relies on `Object.freeze` plus module strict mode (assigning to a frozen property throws `TypeError`). The `as { cents: number }` cast steps around TS's (correct) `readonly` error so the test can prove the *runtime* guard works too.

Type-checking test files needs Node's type definitions. This project doesn't install `@types/node` (hence `tsconfig.json` excludes `*.test.ts`); checking the file alone shows the error a fresh project gets:

```
money.test.ts(2,30): error TS2591: Cannot find name 'node:test'. Do you need to install type
definitions for node? Try `npm i --save-dev @types/node` and then add 'node' to the types field in
your tsconfig.
...
```

In a real Node project: `npm i -D @types/node` and `"types": ["node"]` in `compilerOptions` (listing it is required in TypeScript 7, where `types` defaults to `[]`).

The full suite in `code/` (`node --test` finds all four test files):

```
ℹ tests 31
ℹ suites 8
ℹ pass 29
ℹ fail 0
...
ℹ skipped 1
ℹ todo 1
```

16.15 JSDoc types: TypeScript checking for plain JavaScript

What it is

Without any `.ts` files, TypeScript can check **JavaScript** using types in **JSDoc comments** (`/** @param {number} x */`). Add `// @ts-check` at the top of a file (or `checkJs: true` in `tsconfig.json`) and the editor and `tsc` check it with the same engine — nothing to compile or strip. Svelte and webpack are well-known codebases typed this way.

JSDoc tag	Meaning
`@param {Type} name` / `@returns {Type}`	Parameter and return types
`@type {Type}`	A variable's type
`@typedef {object} Name` + `@property`	Declare a named object type
`@template T`	A generic type parameter
`@import` / `import('./x.js').Type`	Refer to types from other modules

`code/15_jsdoc.mjs`:

```
// @ts-check
/**
 * @typedef {object} LineItem
 * @property {string} sku
 * @property {number} price  unit price in cents
 * @property {number} [qty]  optional, defaults to 1
 */

/**
 * @param {LineItem[]} items
 * @returns {number} total in cents
 */
export function sum(items) {
  return items.reduce((t, i) => t + i.price * (i.qty ?? 1), 0);
}

/** @template T @param {T[]} xs @returns {T | undefined} */
export function last(xs) { return xs[xs.length - 1]; }

/** @type {Record<string, 'admin' | 'user'>} */
const roles = { ana: 'admin', bo: 'user' };
```

Output of `node 15_jsdoc.mjs`:

```
sum  : 1200
last : c
roles: { ana: 'admin', bo: 'user' }
```

The file is part of the project's `tsc --noEmit` check (clean). With mistakes — `code/jsdoc_errors.mjs`:

```
/**
 * @param {number} cents
 * @returns {string}
 */
function formatCents(cents) {
  return (cents / 100).toFixed(2);
}

formatCents('150');                    // string passed where number expected
/** @type {number} */
const count = formatCents(99);         // string result assigned to a number
console.log(count.toUpperCase());      // number has no toUpperCase
```

Output of `tsc --ignoreConfig --noEmit --allowJs --checkJs --strict --target es2024 jsdoc_errors.mjs`:

```
jsdoc_errors.mjs(10,13): error TS2345: Argument of type 'string' is not assignable to parameter of
type 'number'.
jsdoc_errors.mjs(12,7): error TS2322: Type 'string' is not assignable to type 'number'.
jsdoc_errors.mjs(13,19): error TS2339: Property 'toUpperCase' does not exist on type 'number'.
```

Trade-offs

	`.ts` files	JSDoc + `@ts-check`
Syntax	Concise	Verbose comments
Build/strip step	Needed (or Node type stripping)	None
Advanced types	Everything	Most things (complex types can live in a `.d.ts` file)
Adoption	Rename files	Add a comment line per file — easiest first step for an existing JS codebase

TL;DR

`// @ts-check` + JSDoc annotations give TypeScript checking to plain `.js` files with zero build step — a gentle way to adopt types.

Chapter 16 TL;DR

- **Tests** run code and assert results; unit (fast, isolated) → integration → E2E (few). Test behaviour, not internals; exit code gates CI.
- `node:assert/strict`: `equal` (Object.is), `deepEqual` (structure, type-strict), `throws`, `await rejects`, `match`.
- `node:test`: `test`/`it`, `describe`, hooks, subtests, `skip`/`todo`/`only`, `timeout`; `node --test` (finds `*.test.*` and `test/`), `--watch`, reporters.
- **Async**: return/await promises; a missing `await` can make a failing test pass.
- **Doubles**: dependency injection + stubs/fakes first; `t.mock.fn`/`t.mock.method` spies (auto-restored); **fake timers** for time; module mocks need `--experimental-test-module-mocks`.
- **Snapshots** (`t.assert.snapshot`, `--test-update-snapshots`) — review them; **coverage** (`--experimental-test-coverage`) — a hint, not a goal.
- **TypeScript** = JS + static types, erased at runtime; structural typing; `strict`; check with `tsc --noEmit`. **TS 7** (July 2026) is the Go-native compiler, ~10× faster, stricter defaults, `types: []`.

- **`node file.ts`** works (stable type stripping in Node 24) but **never type-checks**. Erasable syntax only (no `enum`, parameter properties, namespaces with code), `.ts` extensions, `import type`.
- Types: literals, unions, tuples, `interface` vs `type`, **`unknown` over `any`**, `never`. **Narrowing** + **discriminated unions** with a `never` exhaustive check.
- **Generics** with constraints and `keyof`; utility types `Partial`, `Pick`, `Omit`, `Record`, `ReturnType`, `Awaited`.
- Classes: `implements`, TS-only `private`/`readonly` vs runtime `#private`; prefer **`as const`** objects to `enum`; **`satisfies`** checks without widening.
- **`*.test.ts`** run directly with `node --test`; type-checking them needs `@types/node`.
- **JSDoc + `// @ts-check`**: type-check plain JavaScript with no build step.

Next: put it all together for interviews — Chapter 17: JavaScript for Interviews.